

DUT-Testbench Connections

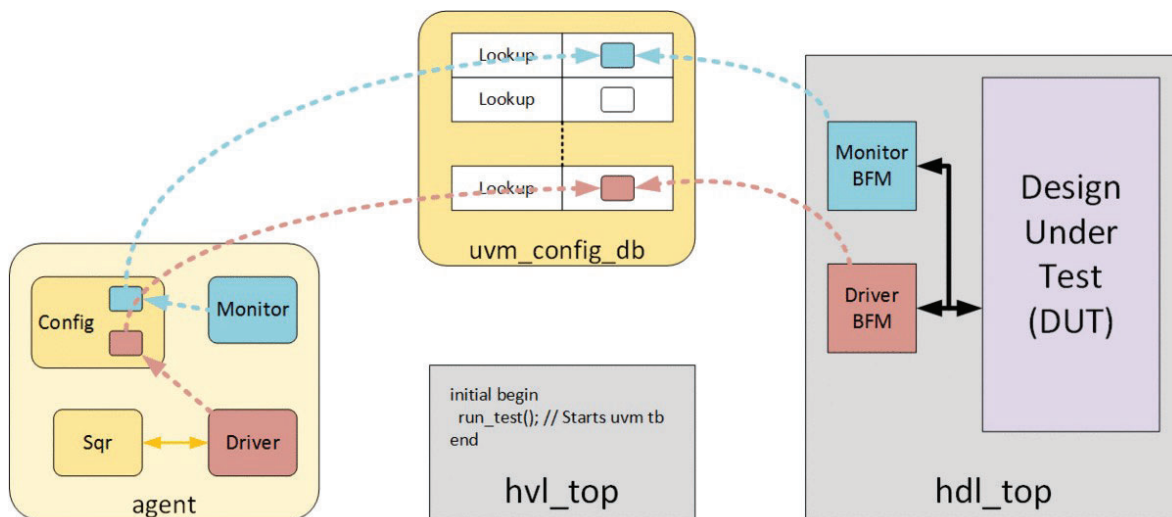
Connecting the Testbench to the DUT

Topic Overview

In this section of the UVM cookbook we consider the problem of connecting the UVM testbench to the RTL DUT.

The UVM testbench objects cannot make a direct connection to the DUT signals in order to drive or sample them. The connection between an agents driver and monitor component objects and the DUT is indirected through one or more BFM components that have static signal ports. These BFM components are implemented either as modules or interfaces and in order to complete the connection to a UVM monitor or driver component class we use a SystemVerilog virtual interface handle to reference the content of a static interface.

The UVM methodology uses the `uvm_config_db` utility to pass a virtual interface handle from a static testbench module to a UVM object class. Two examples of how to do this can be found in the section on using a virtual interface to implement BFMs.



Using the `uvm_config_db` to pass virtual interface handles from `hdl_top` to an agent

Overview showing how concrete BFM interfaces in the `hdl_top` are referenced from the agent monitor and driver UVM objects via virtual interface handles in the `uvm_config_db`

In most examples in the UVM cookbook we use the "Dual Top" methodology. What this means is that the DUT and the BFMs instantiated in one static module hierarchy, the top of which is usually referred to as the "`hdl_top`", and that the part of the testbench that is responsible for starting up the UVM test is in a separate module which is referred to as the "`hvl_top`". This separation helps when there are separate teams working on the design and verification sides of the project, and it also helps when moving to testbench acceleration since at that point the contents of the `hdl_top` will be synthesizable.

We also consider an alternative way of connecting from a SystemVerilog class object to a static BFM using an abstract-concrete class pair. This approach uses polymorphism and a class handle to communicate with a BFM using an API.

Finally we discuss the handling of parameters in DUT-TB connections, where parameters are used to specify the width of signal fields in a protocol interface.

Interfaces and Virtual Interfaces

SystemVerilog Interfaces

The SystemVerilog interface provides a convenient means of organizing related signals into a container in order to simplify connections between modules. An interface can contain:

- Port declarations
- Signal and variable declarations
- Any SystemVerilog code except a module instantiation
- Modport
- Other interfaces

An interface may be declared with or without ports. If it is declared with ports then those ports need to be assigned to signals when the interface is instantiated.

All signals declared as ports of the interface or within the interface can be passed between modules through one interface entity. A module can have interface ports and may mix these with other types of signal ports. Signals within an interface can be referenced or be assigned using a hierarchical reference within the interface – i.e. `interface_name.signal_name`

Interfaces can include any SystemVerilog code except a module instantiation. This means that they can contain initial and always blocks, tasks, functions, SystemVerilog assertions (SVA), covergroups, and classes, all of which are useful for constructing verification components.

SystemVerilog interfaces can also contain modports, which allow the direction of the interface signals to be organised dependent on a usage viewpoint. For instance, a bus master would always drive an address and strobe signal, whilst a bus slave would always receive these signals. Although, from a methodology perspective this appears to be a good idea, there is an element of clumsiness in the language surrounding the modport construct for it not to be particularly useful in practice.

The Virtual Interface

In SystemVerilog a class cannot make a hierarchical reference to a signal without being declared within a module or interface scope where those kinds of references are allowed. For the purposes of developing reusable testbenches using packages, this is very restrictive.

However, SystemVerilog classes defined in a package can reference signals within an interface via an interface handle stored in a virtual interface variable. This allows a class to either assign or sample values of signals inside an interface, or to call tasks or functions within an interface.

```
class protocol_driver extends uvm_driver #(protocol_seq_item);

virtual protocol_interface vif;
// Virtual interface declaration

....

task run_phase(uvm_phase phase);
    @(posedge vif.clock);
    // Accessing the clock signal via the virtual interface variable
    vif.send_packet(...);
    // Calling a BFM method via the virtual interface variable
endtask
```

If a virtual interface is not assigned an interface handle, it has a default value of null and any attempt by the class to access something inside it will result in a null pointer exception in the simulator. For the virtual interface variable to become valid, it must be assigned with an actual interface instance in the testbench module part of the testbench. There are various ways in which this can be done SystemVerilog, but for the UVM the recommended and most scaleable way is to use the `uvm_config_db` to make this assignment.

The UVM Configuration Database (`uvm_config_db`)

The `uvm_config_db`

The `uvm_config_db` is a UVM utility class that is used to pass configuration data objects between component objects in a UVM testbench. Any call to the `uvm_config_db` is parameterised with the type of the data object that is stored or retrieved. To store an object in the `uvm_config_db`, its `set()` method is used, and to retrieve an object from the `uvm_config_db`, its `get()` method is used. Both these methods have arguments that assign the object; associate the object with a lookup key; and define which components within the UVM testbench hierarchy can reference the object.

For more information on the general use of the `uvm_config_db` see the following UVM cookbook page.

Connecting a UVM testbench to a DUT using the `uvm_config_db`

To pass a virtual interface handle from the static module based part of the testbench (`hdl_top`), to a UVM component, use the `uvm_config_db` as follows:

Step 1: In the HDL part of the testbench assign the static interface to a virtual interface entry in the `uvm_config_db`:

```
sfr_if SFR_IF();  
// Declaration of SFR interface - This will be connected to modules  
initial begin  
    uvm_config_db #(virtual sfr_if)::set(null, "uvm_test_top", "SFR", SFR_IF);  
end
```

Note the following:

- The `uvm_config_db` is parameterised with the type *virtual sfr_if*
- The first argument of the `set()` method is context, intended to be assigned a UVM component object handle; in this case since we are in the HDL part of the testbench, a null object handle is assigned.
- The second argument of the `set()` method is a string used to identify the UVM component instance name(s) within the UVM testbench component hierarchy that may access the data object. This is “`uvm_test_top`” here to restrict access to the top level UVM test object. It could have been assigned a wildcard such as “*”, which means that all components in the UVM testbench could access it, but this may not be helpful, and carries a potential lookup overhead in the `get()` process.
- The third argument of the `set()` method is a string, intended as the lookup name, i.e. a string that can be used to uniquely identify the virtual interface from within the `uvm_config_db`.
- The final argument of the `set()` is the static interface assigned to the virtual interface handle entry that is created within the `uvm_config_db`.

Step 2: In a UVM test component assign the virtual interface handle from the uvm_config_db entry to the virtual interface handle inside a configuration object:

```
// Virtual interface handle declared inside an agent configuration
object
class sfr_config extends uvm_object;
...
virtual sfr_if sfr_vif;
...
endclass

class test extends uvm_test;
...

function void build_phase(uvm_phase phase);
    sfr_config cfg = sfr_config::type_id::create("cfg");

    // Assigning the SFR virtual interface handle via the uvm_config_db
    #(...)::get() method
    if(!uvm_config_db #(virtual sfr_if)::get(this, "", "SFR",
    cfg.sfr_vif)) begin
        `uvm_error("VIF_GET", "Unable to find the SFR virtual interface in
    the uvm_config_db", UVM_LOW)
    end
end
```

Note the following:

- The uvm_config_db::get() method is a function that returns a bit value to indicate whether the object retrieval was been successful or not; this is tested to ensure that the testbench does not proceed if the lookup fails.
- The uvm_config_db::get() method is parameterised with the virtual interface type so that the right type of object is retrieved from the database.
- The first argument of the get() method, context, is passed the handle for the UVM component in which the call is being made – this
- The second argument, instance_name, is passed an empty string, "", this means that only the path string for the component is applied based on the component path string derived from the component handle passed in context (i.e. "uvm_test_top").
- The third argument is the uvm_config_db lookup string, this should match the lookup string assigned in the HDL module, in this case "SFR".
- The fourth argument is the virtual interface handle. In this case the handle is inside a uvm_config_object that will be passed to an agent.

If you are reusing a verification component, then all you need to know is the name of its interface and how to assign it to the UVM verification components configuration object. You can then apply the above recipe to make the virtual interface assignments. If you have to create a new verification component then you need to be aware of some implementation considerations.

Virtual Interface BFM

In order to make verification components reusable between testbenches they are organised as `uvm_agents` with an associated signal interface. These are also referred to as UVCs (Universal Verification Components). Inside the `uvm_agent`, there are two types of `uvm_components` that interact with the virtual interface. The driver component is responsible for the active part of the testbench, converting information contained within `sequence_items` into interface activity which involves driving and sampling signal values via a virtual interface handle. The monitor is responsible for the passive side of the testbench, and will sample interface activity via a virtual interface handle to create `sequence_items` that represent interface events.

When designing verification components, there are various ways in which the driver and the monitor can be implemented:

- The interface is a signal container and all signal driving and sampling takes place directly in the UVM driver and monitor classes.
- The interface has methods to either make interface transfers or to sample interface transfers and these are called from the UVM driver and monitor classes.
- A mixture of the two

The simplest approach is to keep all signal accesses within the UVM driver and monitor components and leave the interface as a signal bundle. If the interface is fairly simple, and simulation is the only intended usage for the verification component, then this approach is acceptable.

However, if the potential use models for the verification component are likely to include hardware-assisted acceleration platform using emulation or an FPGA prototyping platform, then the driver and monitor implementations have to be changed so that they are based on calling methods in the interface, rather than direct signal access. This enables the interface to be made synthesisable at some point. In the first instance, an interface might be implemented with methods that use non-synthesisable behavioural code that will only work in a simulator, but in a later second pass these methods can be re-implemented to use synthesisable code without needing to change the UVM driver or monitor code.

Ultimately, the most scalable approach is to determine which parts of the protocol abstraction are best implemented as UVM component code using the strengths of object oriented programming and which are best implemented as a synthesisable task called in the interface. An example of this would be an Ethernet protocol where assembling all the bits in a packet might be best done in the class world, so that sending the packet can be delegated to a relatively simple synthesisable task in the interface.

To illustrate how different implementations might work out in practice, let us consider two alternative ways of implementing the same verification component. In the first example, the interface is a simple signal container and the driver and monitor components reference a virtual interface to drive and monitor interface transfers. In the second example, the interface has tasks to drive and monitor interface transfers and the UVM driver and monitor components reference a virtual interface to call these tasks indirectly.

Both examples separate out the HDL domain of the testbench and the HVL (Hardware Verification Language) or UVM domain of the testbench into two separate modules. This is referred to as the two or dual top testbench architecture.

For testbenches that may eventually need to be co-simulated with an emulator or a FPGA prototype dual domain partitioning^[1] is necessary to group all of the synthesisable code into a separate HDL module hierarchy that can be compiled for a hardware platform. All the non-synthesisable UVM start up code resides in the HVL module. For other testbenches, the dual top approach provides a convenient means of separating out concerns, enables design teams to make changes to the HDL domain without impacting the verification environment, and it enables verification teams to make changes to the HVL domain without impacting the design team.

Example 1 - Two top, simulation only version

In the first example, of an agent only intended for simulation, the protocol interface is declared as a signal bundle and is instantiated in the hdl_top testbench module.

```
interface sfr_if(input clk, input reset);

    logic[7:0] address;
    logic[7:0] write_data;
    logic[7:0] read_data;
    logic we;
    logic re;

endinterface: sfr_if
```

In the driver class of the agent, the conversion of the transaction (sequence_item) content to the clock cycle based stimulus pin wiggles takes place inside the run_phase() task, with the signals either being driven or sampled via the virtual interface handle.

```
task sfr_driver::run_phase(uvm_phase phase);
    sfr_seq_item item;

    forever begin
        seq_item_port.get_next_item(item);
        if(SFR.reset == 1) begin
            SFR.re <= 0;
            SFR.we <= 0;
            SFR.address <= 0;
            SFR.write_data <= 0;
            wait(SFR.reset == 0);
        end
        else begin
            @(posedge SFR.clk);
            SFR.address = item.address;
            SFR.we <= item.we;
            SFR.write_data <= item.write_data;
            SFR.re <= item.re;
            @(posedge SFR.clk);
            if(SFR.re == 1) begin
                item.read_data = SFR.read_data;
                SFR.re <= 0;
            end
            SFR.we <= 0;
        end
        seq_item_port.item_done();
    end

endtask: run_phase
```

In the monitor class of the agent, the monitor samples the interface signals via the virtual interface handle during its `run_phase()` task, and publishes the corresponding analysis transactions (`sequence_items`) when it determines that a transaction has completed.

```
task sfr_monitor::run_phase(uvm_phase phase);
    sfr_seq_item item;

    forever begin
        @(posedge SFR.clk);
        if((SFR.we == 1) || (SFR.re == 1)) begin
            item = sfr_seq_item::type_id::create("item");
            item.we = SFR.we;
            item.re = SFR.re;
            item.address = SFR.address;
            item.write_data = SFR.write_data;
            item.read_data = SFR.read_data;
            ap.write(item);
        end
    end
end

endtask: run_phase
```

The testbench has two top level modules, namely the `hdl_top`, which contains the interface and the DUT, and the `hvl_top` with the initial block for starting the UVM testbench. The following two code snippets represent the important code from these two modules.

"The `hdl_top` module"

```
module hdl_top;

import uvm_pkg::*;

sfr_if SFR(.clk(clk), .reset(reset));

sfr_dut dut (.clk(clk),
             .reset(reset),
             .address(SFR.address),
             .write_data(SFR.write_data),
             .we(SFR.we),
             .re(SFR.re),
             .read_data(SFR.read_data));

initial begin
    uvm_config_db #(virtual sfr_if)::set(null, "uvm_test_top", "SFR",
    SFR);
end

endmodule
```

"The `hvl_top` module"

```

module hvl_top;

import uvm_pkg::*;
import sfr_test_pkg::*;

initial begin
    run_test();
end

endmodule: hvl_top

```

In summary, this example uses only one interface, and all of the signal processing takes place in the class-based UVM testbench domain.

Example 2 - Two top, emulation friendly version

In the second example, there are two separate interfaces, one for driving stimulus (driver_bfm) and one for monitoring the interface (monitor_bfm). For emulation friendliness, the agents signal level driver and monitoring functionality has been moved to the interface as a method, and the UVM agent driver and monitor objects make the method call via the appropriate virtual interface handle ^[1].

"The Driver"

```

task sfr_driver::run_phase(uvm_phase phase);
    sfr_seq_item item;

    forever begin
        seq_item_port.get_next_item(item);
        SFR.execute(item);
        seq_item_port.item_done();
    end
end

```

"The Driver BFM"

```

task execute(sfr_seq_item item);
    if(reset == 1) begin
        wait(reset == 0);
    end
    else begin
        @(posedge clk);
        address = item.address;
        we <= item.we;
        write_data <= item.write_data;
        re <= item.re;
        @(posedge clk);
        if(re == 1) begin
            item.read_data = read_data;
            re <= 0;
        end
        we <= 0;
    end
end

```



```
endtask: execute
```

"The Monitor"

```
task sfr_monitor::run_phase(uvm_phase phase);
    sfr_seq_item item;

    forever begin
        item = sfr_seq_item::type_id::create("item");
        SFR.monitor(item);
        ap.write(item);
    end

endtask: run_phase
```

"The Monitor BFM"

```
task monitor(sfr_seq_item item);
    @(posedge clk);
    if((we == 1) || (re == 1)) begin
        item.we = we;
        item.re = re;
        item.address = address;
        item.write_data = write_data;
        item.read_data = read_data;
    end

endtask: monitor
```

If you compare the two examples, you should note that the functionality is essentially the same, and that the only difference is the delegation of the signal handling to methods in the bfm interfaces. Although, the code in the interfaces is not synthesisable, the API call required to either execute a transaction or to monitor a bus transfer is in place, ready for the bfm interfaces to be replaced with a synthesisable implementation.

In the hdl_top testbench module there are now two interfaces (master_bfm and monitor_bfm) and this requires two virtual interfaces to be set in the uvm_config_db:

```
module hdl_top;

import uvm_pkg::*;

sfr_master_bfm SFR_MASTER(.clk(clk),
                           .reset(reset),
                           .address(address),
                           .write_data(write_data),
                           .we(we),
                           .re(re),
                           .read_data(read_data));

sfr_monitor_bfm SFR_MONITOR(.clk(clk),
                             .reset(reset),
                             .address(address),
```

```
        .write_data(write_data),
        .we(we),
        .re(re),
        .read_data(read_data));

sfr_dut dut (.clk(clk),
            .reset(reset),
            .address(address),
            .write_data(write_data),
            .we(we),
            .re(re),
            .read_data(read_data));

initial begin
    uvm_config_db #(virtual sfr_master_bfm)::set(null, "uvm_test_top",
"SFR_MASTER", SFR_MASTER);
    uvm_config_db #(virtual sfr_monitor_bfm)::set(null, "uvm_test_top",
"SFR_MONITOR", SFR_MONITOR);
end

endmodule
```

The hvl_top testbench module is exactly the same as in example 1.

In summary, the emulation friendly version of the example uses two separate interfaces, and the signal driving and monitoring functionality is initiated via API calls in those interfaces. This example can be used for simulation purposes and provides a bridge to an implementation where the bfm interfaces are replaced with an implementation that would run on an emulator or a hardware prototyping platform.

Most other examples in the UVM cookbook will follow the emulation friendly approach. The exceptions will be those that have been simplified to avoid the use of agents with interfaces in order to illustrate a point.

Handling Parameterization

Parameters are commonly used to configure design IP and interfaces. From the perspective of VIP, parameters usually affect the width of bus fields or the number of channels or lanes in use.

SystemVerilog interfaces may be parameterised and, when they are, their virtual interface handles also need to be parameterised. This impacts the UVM VIP code since the parameter values of the virtual interface inside the monitor and driver of the agent need to match those of the static interface in the `hdl_top` testbench module. Assigning different parameter values to an interface effectively creates a new type.

Handling parameters in RTL is well understood. It usually requires top level parameters to be passed down the design hierarchy, with some possible modification. Parameter can be handled in the same way in class libraries, but this is often inconvenient since in the worst case it means that every sequence item has to be parameterised. If there are several parameterised interfaces of the same type, but with different parameters, then there is scope for error. There are two main strategies that are used to cope with parameters in VIP interfaces:

- Use the maximum possible value of the parameter and only connect the wires and channels used
- Use a parameter package to manage the parameters and typedef parameterised classes

In the example that goes with this article, we use a mixture of the two.

The testbench parameter package

Using a testbench parameter package allows you to manage your testbench parameters. Keeping all the parameter definitions in one package allows you to ensure that there is only one file that needs to be edited should parameters change.

The code example below shows the typical content of a testbench parameter package. The package needs to import all the VIP agent packages so that specialised versions of the parameterised classes can be typedefed. For each interface, a class should be defined that contains the parameter values defined as localparams. This class then provides a scoped container for those parameter values. Then the parameterised classes that the user has to deal are defined as specialised typedefs using the class scope. Finally, the typedefs for the specialised virtual interface handles are declared.

```
package tb_params_pkg;

// Import the VIP package to get access to all the VIP class types
import sfr_agent_pkg::*;

// Class declaration with localparams - this creates a defined
// namespace
// and is useful for handling multiple VIPs of the same type
class SFR;

    localparam sfr_addr_width = 16;
    localparam sfr_data_width = 32;

endclass

// typedefs for those parts of the agent code exposed to the user,
// using the typedefs
```

```
// avoids having to type in the parameters
typedef sfr_config_object #(SFR::sfr_addr_width, SFR::sfr_data_width)
SFR_cfg_t;
typedef sfr_agent #(SFR::sfr_addr_width, SFR::sfr_data_width)
SFR_agent_t;

// Declaration of virtual interface handles
typedef virtual sfr_monitor_bfm #(SFR::sfr_addr_width,
SFR::sfr_data_width) SFR_monitor_bfm_t;
typedef virtual sfr_master_bfm #(SFR::sfr_addr_width,
SFR::sfr_data_width) SFR_master_bfm_t;

endpackage
```

This example only shows the content for one VIP, but when there are multiple VIPs with parameters they should all be grouped in the same testbench parameter package.

Using parameterised interfaces with the uvm_config_db

If you have parameterised interfaces in your hdl_top testbench module you will need to use the uvm_config_db #():set() method with the same parameters. There are two ways to do this, either using the parameters from the tb_params_pkg, or by using a specialised typedef for the virtual interface as defined in the package:

```
import uvm_pkg::*;
import tb_params_pkg::*; // Import of the tb_params_pkg to access
typedefs

logic clk;
logic reset;

wire[SFR::sfr_addr_width-1:0] address;
wire[SFR::sfr_data_width-1:0] write_data;
wire[SFR::sfr_data_width-1:0] read_data;
wire we;
wire re;

// Parameterised version of the sfr_master_bfm using the SFR scope to
define the parameters:
sfr_master_bfm #(.ADDR_WIDTH(SFR::sfr_addr_width),
.DATA_WIDTH(SFR::sfr_data_width))
SFR_MASTER(.clk(clk),

.reset(reset),

.address(address),

.write_data(write_data),

.read_data(read_data),
```

```

.re(re),
.we(we));

sfr_monitor_bfm #(.ADDR_WIDTH(SFR::sfr_addr_width),
                  .DATA_WIDTH(SFR::sfr_data_width))
SFR_MONITOR(.clk(clk),

.reset(reset),

.address(address),

.write_data(write_data),

.read_data(read_data),

.re(re),

.we(we));

initial begin
    // Using a parameterised virtual interface handle:
    uvm_config_db #(virtual sfr_master_bfm
#(.ADDR_WIDTH(SFR::sfr_addr_width),

.DATA_WIDTH(SFR::sfr_data_width)))::set(null, "uvm_test_top",
"SFR_MASTER", SFR_MASTER);

    // Using the virtual interface type declared in the tb_params_pkg:
    uvm_config_db #(SFR_monitor_bfm_t)::set(null, "uvm_test_top",
"SFR_MONITOR", SFR_MONITOR);
end

```

In the UVM test, the virtual interface handle typedef from the `tb_params_pkg` is used to parameterise the `uvm_config_db` `get()` call:

```

class sfr_test extends uvm_component;

sfr_env_config env_cfg;
SFR_cfg_t sfr_agent_cfg; // Typedef from tb_params_pkg

sfr_env env;

extern function void build_phase(uvm_phase phase);
extern task run_phase(uvm_phase phase);

endclass: sfr_test

```

```

function void sfr_test::build_phase(uvm_phase phase);
    env_cfg = sfr_env_config::type_id::create("env_cfg");
    sfr_agent_cfg = SFR_cfg_t::type_id::create("sfr_agent_cfg");
    //
    // Getting the parameterised virtual interface handles from the
    uvm_config_db
    //
    if(!uvm_config_db #(SFR_master_bfm_t)::get(this, "", "SFR_MASTER",
sfr_agent_cfg.SFR_MASTER)) begin
        `uvm_error("BUILD_PHASE", "Unable to find virtual interface
sfr_master_bfm in the uvm_config_db")
    end
    if(!uvm_config_db #(SFR_monitor_bfm_t)::get(this, "", "SFR_MONITOR",
sfr_agent_cfg.SFR_MONITOR)) begin
        `uvm_error("BUILD_PHASE", "Unable to find virtual interface
sfr_master_bfm in the uvm_config_db")
    end
    sfr_agent_cfg.is_active = 1;
    env_cfg.sfr_agent_cfg = sfr_agent_cfg;
    env = sfr_env::type_id::create("env", this);
    env.cfg = env_cfg;
endfunction: build_phase

```

In the env_config and the env classes, the typedefs for the specialised version of the sfr_agent_config and sfr_agent classes are used to pass the parameter values down through the class hierarchy. Using the tb_params_pkg definitions ensures that all the class hierarchy parameterisation is consistent.

```

// The sfr_env_config class:
class sfr_env_config extends uvm_object;

SFR_cfg_t sfr_agent_cfg;

endclass: sfr_env_config

// The sfr_env
class sfr_env extends uvm_component;

sfr_env_config cfg;
sfr_scoreboard sb;
SFR_agent_t agent; // Using the SFR_agent_t typedef from tb_params_pkg

extern function void build_phase(uvm_phase phase);
extern function void connect_phase(uvm_phase phase);

endclass: sfr_env

function void sfr_env::build_phase(uvm_phase phase);
    if(cfg == null) begin

```

```

        if(!uvm_config_db #(sfr_env_config)::get(this, "", "CFG", cfg))
begin
    `uvm_error("BUILD_PHASE", "Unable to find environment configuration
object in the uvm_config_db")
end
end
sb = sfr_scoreboard::type_id::create("sb", this); agent =
SFR_agent_t::type_id::create("agent", this); agent.cfg = cfg.sfr_agent_cfg;
endfunction: build_phase

// And the sfr_agent - to show how the parameters are passed through to
the driver.
// Note that the parameters are defined more generically here to ensure
that the agent can be reused
//
class sfr_agent #(SFR_ADDR_WIDTH = 8, SFR_DATA_WIDTH = 8) extends
uvm_component;

typedef sfr_agent #(SFR_ADDR_WIDTH, SFR_DATA_WIDTH) this_t;

`uvm_component_param_utils(this_t) uvm_analysis_port

#(sfr_seq_item) ap; uvm_sequencer #(sfr_seq_item)

sequencer;

sfr_driver #(SFR_ADDR_WIDTH, SFR_DATA_WIDTH) driver; sfr_monitor
#(SFR_ADDR_WIDTH, SFR_DATA_WIDTH) monitor;

sfr_config_object #(SFR_ADDR_WIDTH, SFR_DATA_WIDTH) cfg;

function new(string name = "sfr_agent", uvm_component parent = null); super.new(name, parent);
endfunction

extern function void build_phase(uvm_phase phase);
extern function void connect_phase(uvm_phase phase);

endclass: sfr_agent

function void sfr_agent::build_phase(uvm_phase phase);
    if(cfg == null) begin
        if(!uvm_config_db #(sfr_config_object #(SFR_ADDR_WIDTH,
SFR_DATA_WIDTH))::get(this, "", "SFR_CFG", cfg)) begin
            `uvm_error("BUILD_PHASE", "Unable to find sfr agent config object in the uvm_config_db")
        end
    end
end

```

```

        end
    end
    ap = new("ap", this);
    monitor = sfr_monitor #(SFR_ADDR_WIDTH,
SFR_DATA_WIDTH)::type_id::create("monitor", this);
    if(cfg.is_active == 1) begin
        driver = sfr_driver #(SFR_ADDR_WIDTH,
SFR_DATA_WIDTH)::type_id::create("driver", this);
        sequencer = uvm_sequencer
        #(sfr_seq_item)::type_id::create("sequencer", this);
    end
endfunction: build_phase

```

Finally, in order to ensure that any sequences can be written without having to deal with parameters, we have taken the "max width" approach to the sequence_item item definition. The maximum width of the address and data buses is 32 bits, so we have used this to size the the address and data fields in the sequence item. Any unused bits will be ignored.

```

class sfr_seq_item extends uvm_sequence_item;

`uvm_object_utils(sfr_seq_item)

function new(string name = "sfr_seq_item");
    super.new(name);
endfunction

rand bit[31:0] address;
rand bit[31:0] write_data;
rand bit we;
rand bit re;

bit[31:0] read_data;

endclass: sfr_seq_item

```

The reason that the UVM classes have to take on parameters is that the virtual interface handles have to be declared and assigned using parameters, an alternative solution to this problem is to use the abstract-concrete class implementation approach.

Abstract-Concrete Class Connections

Note: This approach to DUT-TB connections is a simulation only technique

An alternative to using a virtual interface handle for DUT to UVM testbench connections is to use an abstract concrete class pair. The abstract version of the class defines templates for all the methods that are available in the BFM. The concrete version of the class is declared in the BFM and it extends the abstract version, providing an implementation of the methods. The handle of the concrete class is assigned to the abstract handle in the agent classes via the `uvm_config_db`. The properties of class object polymorphism allow the abstract class handle to interact with the BFM using the method implementations in the concrete version of the class.

If a class is defined inside an interface or a module, it can reference any of the variables within the interface or module scope. This can be used to simplify the handling of parameters.

The following code snippets show the Abstract and Concrete versions of the `sfr_master` class:

```
// The abstract class for the SFR master driver/BFM
class sfr_master_abstract;

virtual task execute(sfr_seq_item item);

endtask

endclass

// The implementation for the SFR master BFM, containing the concrete
implementation of the sfr_master
module sfr_master_bfm #(ADDR_WIDTH = 8,
                        DATA_WIDTH = 8)
    (input clk,
     input reset,
     output logic[ADDR_WIDTH-1:0] address,
     output logic[DATA_WIDTH-1:0] write_data,
     input logic[DATA_WIDTH-1:0] read_data,
     output logic we,
     output logic re);

import uvm_pkg::*;
import sfr_agent_pkg::*;

always @(reset or posedge clk) begin
    if(reset == 1) begin
        re <= 0;
        we <= 0;
        address <= 0;
        write_data <= 0;
    end
end

// Definition of the concrete version of the sfr_master class:
```

```

class sfr_master_concrete extends sfr_master_abstract;

    task execute(sfr_seq_item item);
        if(reset == 1) begin
            wait(reset == 0);
        end
        else begin
            @(posedge clk);
            address = item.address;
            we <= item.we;
            write_data <= item.write_data;
            re <= item.re;
            @(posedge clk);
            if(re == 1) begin
                item.read_data = read_data;
                re <= 0;
            end
            we <= 0;
        end
    endtask: execute
endclass

// Declaration of the SFR_MASTER class handle
sfr_master_concrete SFR_MASTER;

// Initial block, constructing the SFR_MASTER object and assigning the
// handle into the uvm_config_db
// Note the use of the sfr_master_abstract handle as the uvm_config_db
// parameter
initial begin
    SFR_MASTER = new();
    uvm_config_db #(sfr_master_abstract)::set(null, "uvm_test_top",
    "SFR_MASTER", SFR_MASTER);
end

endmodule: sfr_master_bfm

```

Note that in the above code, the BFM has been changed to a module from an interface, this is another freedom that using the abstract-concrete pattern enables. Modules allow the BFM to have hierarchy.

Inside the SFR agent classes, any reference to the BFM is via handles of the `sfr_master_abstract` type:

```

class sfr_driver extends uvm_driver #(sfr_seq_item);

    // Abstract version of the sfr_master class:
    sfr_master_abstract SFR;

    extern task run_phase(uvm_phase phase);

```

```

endclass: sfr_driver

task sfr_driver::run_phase(uvm_phase phase);
    sfr_seq_item item;

    forever begin
        seq_item_port.get_next_item(item);
        SFR.execute(item);
        seq_item_port.item_done();
    end

endtask: run_phase

```

The handle to the `sfr_master_abstract` object is passed via the agent config object via the `uvm_config_db`, note that in the following code snippet, the `sfr_monitor` uses the same connection pattern:

```

class sfr_test extends uvm_component;

    sfr_env_config env_cfg;
    sfr_config_object sfr_agent_cfg;

    sfr_env env;

    extern function void build_phase(uvm_phase phase);
    extern task run_phase(uvm_phase phase);

    endclass: sfr_test

    function void sfr_test::build_phase(uvm_phase phase);
        env_cfg = sfr_env_config::type_id::create("env_cfg");
        sfr_agent_cfg = sfr_config_object::type_id::create("sfr_agent_cfg");

        // Assigning the handle to the concrete implementations of the
sfr_master and sfr_monitor classes to their abstract counterparts
        if (!uvm_config_db #(sfr_master_abstract)::get(this, "", "SFR_MASTER",
sfr_agent_cfg.SFR_MASTER)) begin
            `uvm_error("BUILD_PHASE", "Unable to find virtual interface
sfr_master_bfm in the uvm_config_db")
        end
        if (!uvm_config_db #(sfr_monitor_abstract)::get(this, "",
"SFR_MONITOR", sfr_agent_cfg.SFR_MONITOR)) begin
            `uvm_error("BUILD_PHASE", "Unable to find virtual interface
sfr_master_bfm in the uvm_config_db")
        end

        sfr_agent_cfg.is_active = 1;
        env_cfg.sfr_agent_cfg = sfr_agent_cfg;

```

```
env = sfr_env::type_id::create("env", this);
env.cfg = env_cfg;
endfunction: build_phase
```

When the abstract-concrete class connection pattern is used, the agent does not have to be parameterised, since class handles are used to make the connection.

In summary, the abstract concrete class approach works by passing a class object handle from the hdl_top part of the testbench to the UVM testbench.

Parameterized Tests

Introduction

SystemVerilog provides a number of ways to pass changeable values through different code structures. Some changeable values must be fixed at elaboration time and others can be changed at run-time after starting a simulation. Changeable values fixed at elaboration time are represented using either a SystemVerilog parameter or `define macro. The use of `define macros causes complications in cases of multiple instances of a module or interface where each instance needs a different changeable value. SystemVerilog parameters are more fitting and the preferred mechanism to pass changeable "type" values as well as to specify bit vector sizes such as data and address widths.

The values of "size" parameters used in the design to control widths are generally required also in the testbench. The remainder of this article illustrates a method to share parameters through parameterized UVM tests inside the dual top testbench architecture.

DVCon Papers

The information in this article is drawn from a conference paper with Xilinx at DVCon 2011 ^[1], and a subsequent updated paper at DVCon 2016 ^[2] fit to the dual top testbench architecture. The material in this article is a result of a collaboration between Mentor and Xilinx.

Registering Parameterized Classes with the String-based Factory

Parameterized classes use the `uvm\_component\_param\_utils and `uvm_object_param_utils macros to register with the UVM factory. The UVM actually has two factories - one string-based and one type-based. These param_utils macros only perform registration with the type-based factory.

For example, given a parameterized test class named `alu_basic_test #(DATA_WIDTH)`, the macro call ``uvm_component_param_utils(alu_basic_test #(DATA_WIDTH))` would expand to:

```
typedef uvm_component_registry #(alu_basic_test #(DATA_WIDTH)) type_id;
// <-- Default null string used for unspecified 2nd string parameter

static function type_id get_type();
    return type_id::get();
endfunction

virtual function uvm_object_wrapper get_object_type();
    return type_id::get();
endfunction
```

The typedef in the code above creates a specialization of the `uvm_component_registry` type, but that type takes two parameter arguments - the first is the type being registered (*alu_basic_test* #(DATA_WIDTH) in this case) with the type-based factory, and the second is the string name that will be used to uniquely identify that type in the string-based registry. Since the ``uvm_component_param_utils` macro does not provide a value for the second parameter, it defaults to the null string and no string-based registration is performed.

Occasionally, you might want to use the string-based factory to create a component (or object). The most common case where the string-based factory is used is during the call to `run_test()`. The `run_test()` call uses either its string argument or the string value from the `+UVM_TESTNAME` plusarg to request a component from the string-based factory.

Since a parameterized UVM component is not registered with the string-based factory by default (per above example), you will need to implement a string-based registration for your top-level test classes so that they can be instantiated by `run_test()`. To accomplish this, you must manually code the actions that the ``uvm_component_param_utils` macro performs.

To perform a string-based registration, you need to provide a string for the second parameter argument that will be unique for each specialization of the test class. You can rewrite the typedef to look like:

```
typedef uvm_component_registry #(alu_basic_test #(DATA_WIDTH),
"basic_test1") type_id; // <-- Unique string specified as 2nd parameter
```

In addition, you must declare a "dummy" specialization of the parameterized test class so that the string name specified above is tied to the particular parameter values.

```
module hvl_top();
    parameter DATA_WIDTH = 4;

    // Associate the string "basic_test1" with the value of DATA_WIDTH
    typedef alu_basic_test #(DATA_WIDTH) dummy;

    initial begin
        run_test("basic_test1");
    end
endmodule
```

Note: Instead of a name like "basic_test1" above, you could use the macro described below to generate a string name like "basic_test_#(8)" with the actual parameter values as part of the string.

Sharing Parameters with Dual Tops

As shown above the `hvl_top` module contains a parameter `DATA_WIDTH` defined in the module. Looking at the `hdl_top`, it also has a parameter called `DATA_WIDTH`.

```
module hvl_top();
    parameter DATA_WIDTH = 4;

    //Pin and BFM interface instantiations using DATA_WIDTH

    alu_rtl #(DATA_WIDTH) dut ( /* port connections */ );

    ...
endmodule
```

```
endmodule
```

Hence if this parameter changes, updates are required in two locations. This can be overcome by using a shared parameter package to define the parameters in a single package which is then imported into both the hvl_top and hdl_top.

Maintaining Consistency in Parameter Lists

Many SystemVerilog parameters can naturally be grouped together in a conceptual "parameter list". These parameters tend to be declared together and are used in many places in the test environment.

Any change to a parameter list, such as adding a new parameter, often requires careful editing of many different classes in many different files, and is an error-prone process.

An optional technique that can be employed is to create a set of macros that can reduce the chance of errors and enforce consistency.

Declaration	<code>`define params_declare #(int BUS_WIDTH = 16, int ADDR_WIDTH = 8)</code>
Instantiation / Mapping	<code>`define params_map #(.BUS_WIDTH(BUS_WIDTH), .ADDR_WIDTH(ADDR_WIDTH))</code>
String Value	<code>`define params_string \$sformat("#(%1d, %1d)", BUS_WIDTH, ADDR_WIDTH)</code>

These macros keep with the reuse philosophy of minimizing areas of change. By using the macros, there is one, well-defined place to make changes in parameter lists.

References

- [1] <https://verificationacademy.com/resource/6648>
- [2] http://events.dvcon.org/2016/proceedings/papers/08_4.pdf